

Streamlining IT Procurement: Automating Standard Laptop Orders

Project Description:

This project builds a simple web-based system that standardizes laptop requests and automates the approval, ordering and tracking process. Employees submit a laptop request by choosing from a catalog of standard models, approvers review and approve or reject each request, and the procurement team converts approved requests into standard laptop orders and tracks them until delivery. A dashboard shows pending, approved, ordered and completed requests, users receive email updates when a status changes, and basic reports summarize requests, approvals and orders.

The system is built with Flask and a MySQL database, with a browser interface made using HTML, CSS and JavaScript. Role-based login keeps each screen limited to the employee, approver/admin or procurement user who needs it. By replacing manual requests and follow-ups with one shared workflow, the platform saves time, reduces manual work and makes request status easy to see.

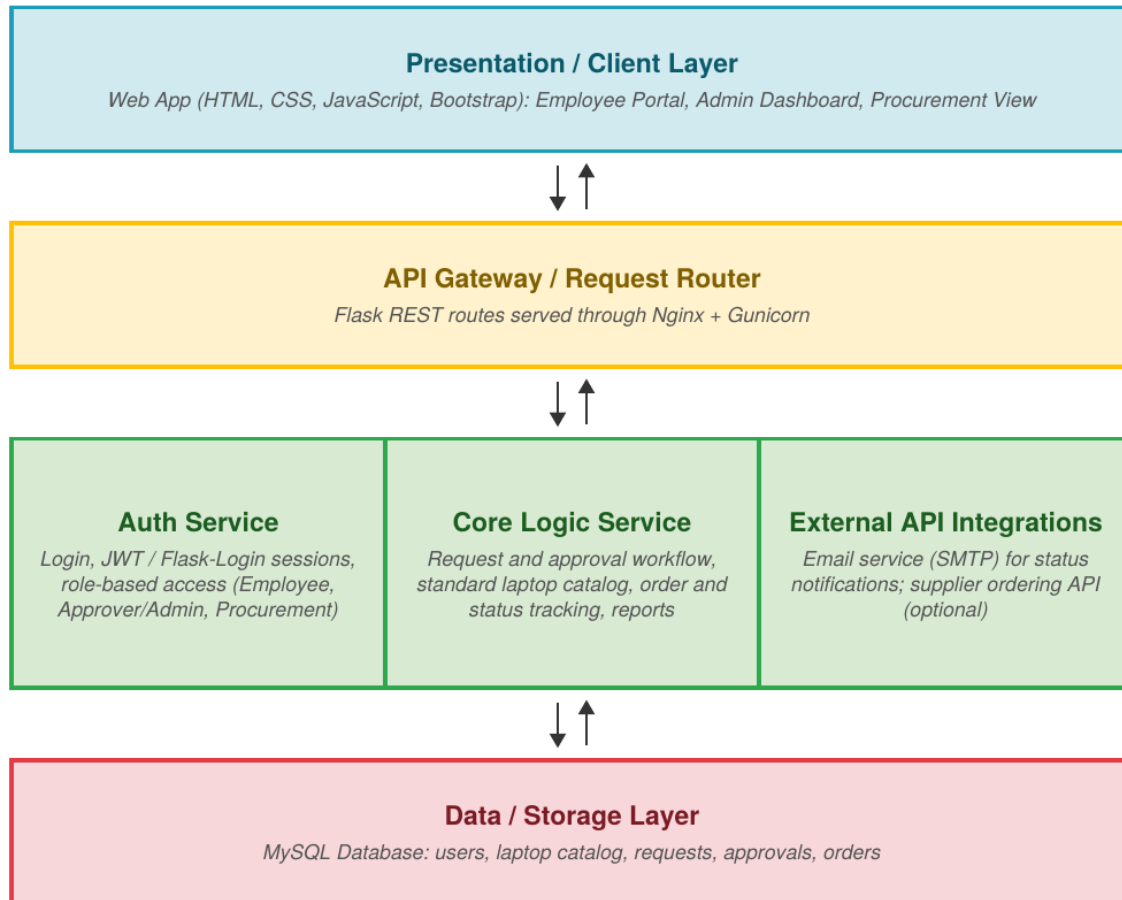
Scenarios

Scenario 1: A new employee joins a company and needs a laptop. They log in, pick a standard model from the catalog, enter their details and submit the request. They can check the status of the request at any time instead of chasing the admin by email.

Scenario 2: An approver opens the list of pending requests, reviews the requested model and the reason, and approves one request and rejects another with a comment. The decision is saved in the approval history and the requester is notified by email.

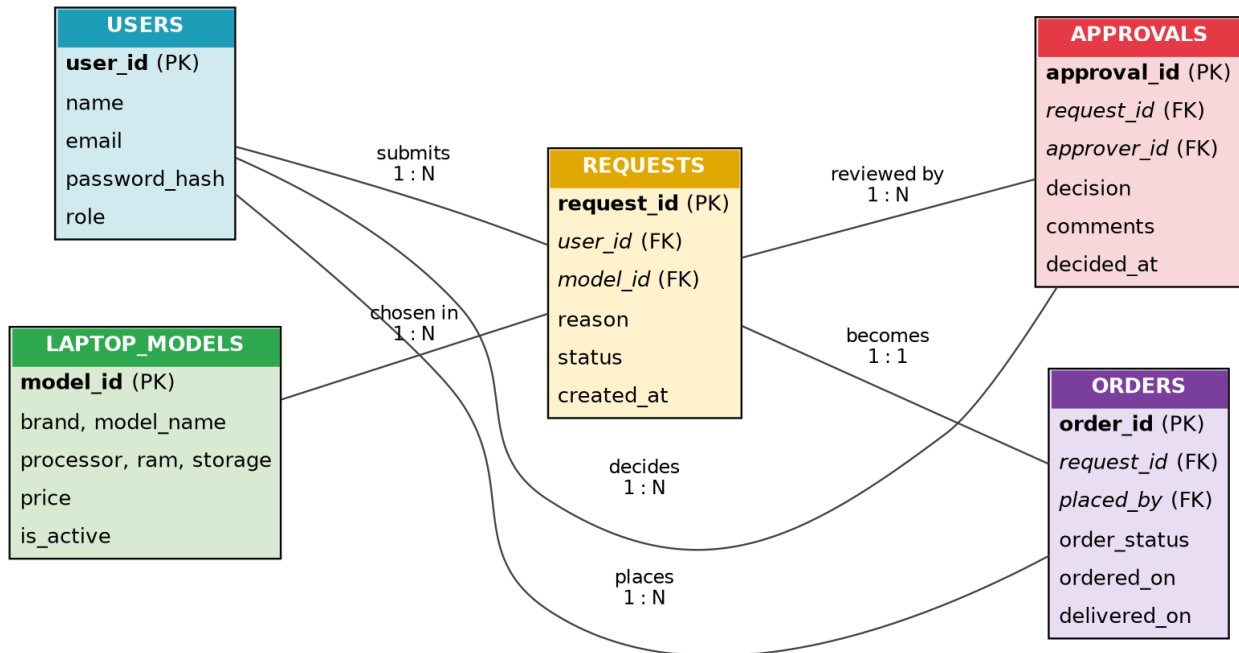
Scenario 3: The procurement team sees the approved requests, converts them into standard laptop orders and updates each order from ordered to shipped to delivered. The admin dashboard and reports show how many requests are pending, ordered and completed.

Technical Architecture:



Description: The system uses a layered architecture. The presentation layer is a browser-based web app (HTML, CSS, JavaScript) with screens for employees, approvers and procurement staff. Requests pass through a Flask routing layer served by Nginx and Gunicorn, which sends them to three services: the Auth Service for login and role-based access, the Core Logic Service for the request, approval, catalog, order and reporting workflow, and the External API Integrations for email notifications and an optional supplier ordering API. All data is stored in a MySQL database.

ER Diagram:



Description: **USERS** stores every person who logs in, with a role (employee, approver/admin or procurement). **LAPTOP_MODELS** is the standard catalog. A user submits many **REQUESTS**, and each request chooses one laptop model. Each request can have **APPROVALS** (decision, comment and the approver who made it), and an approved request becomes one **ORDER**, placed by a procurement user, whose status is tracked until delivery.

Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **MySQL Database Basics:** [MySQL Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Flask Installation Guide](#)

Project Workflow:

Milestone 1: Requirement Analysis and Architecture

- **Activity 1.1:** Study the manual laptop procurement process and list the users and their needs.
- **Activity 1.2:** Select the technology stack: Flask, MySQL and a browser-based interface.
- **Activity 1.3:** Define the architecture and database design, showing how the screens, services and data connect.
- **Activity 1.4:** Set up the development environment, installing the libraries and dependencies for Flask and MySQL.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core features: request submission, approval workflow, standard laptop catalog, order and status tracking, dashboard, notifications and reports.
- **Activity 2.2:** Implement the Flask backend to manage routing, validation and database access.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, creating the routes for each feature.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS and JavaScript with a responsive layout.
- **Activity 4.2:** Create dynamic templates with Flask's render_template to show data based on the logged-in role.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by setting up the environment, the database and the dependencies.
- **Activity 5.2:** Run and test the application locally, and optionally host it online.

Milestone 6: Conclusion

Milestone 1: Requirement Analysis and Architecture

This milestone focuses on understanding the laptop procurement problem and choosing how the system will be built. It covers who uses the system, what each user needs, and how the screens, services and database fit together.

Activity 1.1: Study the Process and Users

- **Employee / Requester:** submits a laptop request and checks its status.
- **Approver / Admin:** reviews requests and approves or rejects them.
- **Procurement / Company:** places standard laptop orders and tracks them.
- **Problem:** manual requests, approval follow-ups, ordering and tracking take time and make request status hard to monitor.

Activity 1.2: Select the Technology Stack

- **Backend:** Python with the Flask framework, with Flask-Login for sessions and role-based access.
- **Database:** MySQL, accessed through Flask-SQLAlchemy.
- **Frontend:** HTML, CSS, JavaScript and Bootstrap.
- **Notifications:** Flask-Mail with SMTP for status emails.

Activity 1.3: Define the Architecture

- **Frontend functionality:** employees browse the catalog and submit requests, approvers review them, procurement staff manage orders, and admins view the dashboard and reports.
- **Backend responsibilities:** Flask routes validate input, check the user's role, run the workflow and read or write the MySQL database.
- **Integration points:** the Core Logic Service triggers an email through the notification service whenever a status changes.

Activity 1.4: Set Up the Development Environment

- **Install Python and pip:** ensure Python 3.8+ is installed along with pip.
- **Create a virtual environment:** `python -m venv venv`, then activate it.
- **Install dependencies:** `pip install -r requirements.txt` (Flask, Flask-SQLAlchemy, Flask-Login, Flask-Mail, pandas).
- **Configure the environment:** create a `.env` file with the database and email settings and keep it out of the repository using `.gitignore`.
- **Set up the folder structure:** `app.py`, `models.py`, `templates`, `static`, `database`, `tests` and `docs`.

[Add screenshot: project folder structure in VS Code]

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the Core Features

The core features are built in the order of the project priorities.

- **Request and approval workflow:** employees submit a request from the catalog, and approvers approve or reject it with a comment. Approval history is kept.
- **Order and status tracking:** approved requests become standard laptop orders whose status moves from ordered to shipped to delivered.
- **Standard laptop catalog:** admins add and update a fixed list of laptop models and specifications so choices stay consistent.
- **Admin dashboard:** shows the number of pending, approved, ordered and completed requests.
- **Notifications:** emails are sent to the requester and admin when a request or order status changes.
- **Reports:** basic reports of requests, approvals and orders.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in app.py for registration, login, the catalog, requests, approvals, orders, the dashboard and reports.

Process User Input:

- Create forms in the templates for registration, login and the laptop request.
- Validate required fields and check that a user's role allows the action before processing.
- Save every request, decision and order in the MySQL database.

Integrate Notifications:

- After a request is approved, rejected or its order status changes, call the mail service to email the requester.

Milestone 3: App.py Development

Milestone 3 focuses on the core logic of app.py, which connects the screens to the workflow and the database. Each feature is exposed through a Flask route, and each route is tested to make sure it behaves as expected.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

Route	Purpose
/register, /login, /logout	Create an account, sign in and sign out.
/catalog	List standard laptop models (admins can add and edit).
/request/new, /my-requests	Submit a laptop request and check its status.
/approvals	View pending requests and approve or reject them.
/orders	Create orders from approved requests and update order status.
/dashboard, /reports	Show request counts by status and basic reports.

Set Up Route Handlers for Each Feature:

- Read the submitted form data, validate it, and return an error message if a required field is missing.
- Allow each route only for the roles that need it (employee, approver/admin, procurement).
- Update the request status as it moves from submitted to approved or rejected, then ordered and completed.
- Render the matching template with the data from the database.

[Add screenshot: app.py routes in the code editor]

Milestone 4: Frontend Development

Milestone 4 focuses on a simple, clear interface so that non-technical users can request and track laptops without training. It uses responsive HTML and CSS together with JavaScript for a smooth experience.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Create a base template with the navigation bar, page title and message area, and extend it for every page.
- Use semantic HTML elements so the pages are organized and accessible.
- Use Bootstrap for consistent forms, tables and buttons.

Design a Responsive Layout Using CSS:

- Create a stylesheet (static/css/style.css) with a clean layout and a clear color for each request status.
- Use flexbox and grid layouts for the dashboard cards and request tables.
- Add media queries so the pages work on desktop, tablet and mobile screens.

Create Separate UI Components:

- **Catalog cards** showing each laptop model with its specifications.
- **Request form** for choosing a model and entering the reason.
- **Status tables** for requests, approvals and orders with status badges.
- **Dashboard cards** for pending, approved, ordered and completed requests.

Activity 4.2: Creating Dynamic Content Rendering

- Use Flask's render_template with Jinja to show pages based on the logged-in role.
- Use JavaScript to confirm actions such as rejecting a request and to filter the request tables.

Milestone 5: Deployment

In Milestone 5, the application is deployed locally to verify correct operation, and can then be hosted online if the team chooses to.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a terminal in the project folder.
- Create and activate a virtual environment, then install all required dependencies from requirements.txt:

```
python -m venv venv  
venv\Scripts\activate  
pip install -r requirements.txt
```

Configure Environment Variables and the Database:

- Copy .env.example to .env and set the database connection and the email (SMTP) settings.
- Run database/schema.sql and then database/seed.sql in MySQL to create the tables and load the sample laptop catalog.

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

```
python app.py
```

- Open a browser and go to `http://127.0.0.1:5000` to use the app.
- Test the full flow with each role: an employee submits a request, an approver approves or rejects it, and procurement creates the order and updates its status.
- Check that the dashboard counts, the email notifications and the reports show the correct results.

[Add screenshot: terminal showing the Flask server running]

Exploring the Website's Web Pages:

Login / Registration Page:

[Add screenshot: login page]

Description: Lets users create an account and sign in. After login, each user is taken to the screens for their role (employee, approver/admin or procurement).

Laptop Catalog Page:

[Add screenshot: catalog page]

Description: Shows the standard laptop models with their specifications. Employees choose a model from here, and admins can add or update models.

Request Page:

[Add screenshot: request form and my requests]

Description: A form where an employee selects a standard model, enters their details and the reason, and submits the request. A table below lists their requests and current status.

Approvals Page:

[Add screenshot: approvals page]

Description: Shows pending requests for the approver, with buttons to approve or reject and a comment box. The approval history is listed for admins.

Orders Page:

[Add screenshot: orders page]

Description: Lists approved requests for procurement, lets them create a standard laptop order and update its status from ordered to shipped to delivered.

Dashboard and Reports Page:

[Add screenshot: dashboard page]

Description: Shows cards for pending, approved, ordered and completed requests, plus basic reports of requests, approvals and orders.

Conclusion:

This project delivers a simple, organized laptop procurement system built with Flask and MySQL that replaces manual requests and follow-ups with one shared workflow. Employees can request standard laptops and see their status, approvers can review requests quickly, and procurement can place and track standard orders. The result reduces manual work, saves time, improves visibility of request status and supports standardized laptop ordering. Looking ahead, the system can be extended with supplier API integration for automatic ordering, support for other IT hardware, multi-level approvals and richer reports, growing from a laptop request tool into a complete IT procurement platform.